

Date : 15-04-2026

```
#
#frozen Set
"""
a={1,2,3}
print(a)
a=frozenset({1,2,3})
print(a)
"""

#
"""
a={1}
print(a)

#a=frozenset(1) # error
#print(a)
"""

#
"""
a={"m"}
print(a)

a=frozenset("m")
print(a)
"""

#
"""
#a={1,2,3} # -- error
#print(a)

a=frozenset((1,2))
print(a)
"""

#
"""
#a={{1,2}:"Bleach"}
#print(a) -- error

a={frozenset({1,2}):"Naruto"}
print(a)
"""

# Synchronous Programming -- normal programs are synchronous in nature

# Tasks run one after another blocking each other untill completion of each task

# Asynchronous Programming

# That task can start before the previous task finishes and they can run concurrently without blocking each other

# asyncio -- Asynchronous I/O
# async def func(): -- async coroutine -- async keyword is used to define an asynchronous function
# await asyncio.sleep(2) -- how much time the function should wait
```

```
# asyncio.run(func()) -- to run the asynchronous function
```

```
# Syntax of Synchronous Programming
```

```
"""
import time
def func():
    print("rasengan")
    time.sleep(2)
    print("chidori")
```

```
func()
"""
```

```
#
```

```
import asyncio
from ntpath import join
"""
```

```
async def func():
    print("rasengan")
    await asyncio.sleep(2)
    print("chidori")
```

```
asyncio.run(func())
"""
```

```
#
```

```
#
```

Date : 16-04-2026

```
# normal synchronous programming
```

```
"""
import time
def func(name,delay):
    print(f"{name} ordered")
    time.sleep(delay)
    print(f"{name} placed")
def main():
    func("tea",4)
    func("juice",3)
    func("vadai",2)
```

```
start=time.time()
main()
end=time.time()
print(f"Total time taaken : {end-start} seconds")
"""
```

```
# Asynchronous programming - gathers multiple coroutines and runs them concurrently without blocking each other
```

```

"""
import asyncio,time

async def func(name,delay):
    print(f"{name} ordered")
    await asyncio.sleep(delay)
    print(f"{name} placed")

async def main():
    await asyncio.gather(func("tea",4),func("juice",3),func("vadai",2))

start=time.time()
asyncio.run(main())
end=time.time()
print(f"Total time taken : {end-start} seconds")
"""

```

create_task() -- to create a task and schedule it to run concurrently with other tasks

```

"""
import asyncio,time

async def func(name,delay):
    print(f"{name} ordered")
    await asyncio.sleep(delay)
    print(f"{name} placed")

async def main():
    t1=asyncio.create_task(func("tea",4))
    t2=asyncio.create_task(func("juice",3))
    t3=asyncio.create_task(func("vadai",2))

    print("Orders taken")

    await t1
    await t2
    await t3

start=time.time()
asyncio.run(main())
end=time.time()
print(f"Total time taken : {end-start} seconds")
"""

```

await is the keyword used to wait for the completion of a coroutine or a task, it can only be used inside an async function, it allows other tasks to run while waiting for the completion of the awaited task, it does not block the entire program but only the current coroutine or task that is awaiting.

```

"""
import asyncio,time

async def func(name,delay):
    print(f"{name} ordered")
    await asyncio.sleep(delay)
    print(f"{name} placed")

```

```

async def main():
    await asyncio.create_task(func("tea",4)) # here it works like synchronous programming because
    we are awaiting each task immediately after creating it, so the next task will not start until the
    previous one finishes
    await asyncio.create_task(func("juice",3))
    await asyncio.create_task(func("vadai",2))
    start=time.time()
    asyncio.run(main())
    end=time.time()
    print(f"Total time taken : {end-start} seconds")

```

async timeout

```

import asyncio,time

```

```

async def func():
    await asyncio.sleep(2)
    print("Task completed")
    async def main():
        t1=asyncio.wait_for(func(),timeout=3) # if the task takes more than 3 seconds to complete, it
        will raise a TimeoutError
        await t1
    asyncio.run(main())

```

```

#

```

```

async def func():
    await asyncio.sleep(4)
    print("Task completed")
    async def main():
        t1=asyncio.wait_for(func(),timeout=3) # if the task takes more than 3 seconds to complete, it
        will raise a TimeoutError
        await t1

```

with try except block TimeoutError can be handled and the program can continue to run without crashing

```

try:
    asyncio.run(main())
except asyncio.TimeoutError:
    print("Task timed out")
else:
    print("Task completed successfully")
finally:
    print("Program finished")

```

without handling the TimeoutError, the program will crash and the finally block will not be executed

```

try:
    asyncio.run(main())
except:
    print("Task timed out")

```

```

else:
    print("Task completed successfully")
finally:
    print("Program finished")

'''

import asyncio,time
# one by one put and get data from the queue
'''

async def put_data(q):
    for i in range(5):
        await asyncio.sleep(1) # simulating the time taken to put data into the queue
        await q.put(i) # to put data into the queue
        print(f"Putting data {i} into the queue")

    async def get_data(q):
        while True:
            await asyncio.sleep(1) # simulating the time taken to get data from the queue
            print(f"Getting data {await q.get()} from the queue")
            q.task_done() # to indicate that the task is done and the queue can move on to the next task
        async def main():
            q=asyncio.Queue() # to create a queue for communication between coroutines
            t1=asyncio.create_task(put_data(q))
            t2=asyncio.create_task(get_data(q))

            await t1
            await q.join()

            t2.cancel()

        asyncio.run(main())
'''

# first put all the data into the queue and then get all the data from the queue and then cancel the
get_data task
'''

async def put_data(q):
    for i in range(5):
        await asyncio.sleep(1) # simulating the time taken to put data into the queue
        await q.put(i) # to put data into the queue
        print(f"Putting data {i} into the queue")

    async def get_data(q):
        while True:
            await asyncio.sleep(1) # simulating the time taken to get data from the queue
            print(f"Getting data {await q.get()} from the queue")
            q.task_done() # to indicate that the task is done and the queue can move on to the next task
        async def main():
            q=asyncio.Queue() # to create a queue for communication between coroutines
            t1=asyncio.create_task(put_data(q))
            await t1
            t2=asyncio.create_task(get_data(q))

```

```
await q.join()
print("All data processed and get_data task cancelled")
t2.cancel() # WITHOUT THIS: Program hangs forever! WITH THIS: Program exits cleanly
```

```
asyncio.run(main())
```

```
'''
```

```
# Another Method
```

```
'''
```

```
async def put_data(q):
```

```
for i in range(5):
```

```
    await asyncio.sleep(1) # simulating the time taken to put data into the queue
```

```
    await q.put(i) # to put data into the queue
```

```
    print(f"Putting data {i} into the queue")
```

```
async def get_data(q):
```

```
while True:
```

```
    await asyncio.sleep(1) # simulating the time taken to get data from the queue
```

```
    print(f"Getting data {await q.get()} from the queue")
```

```
    q.task_done() # to indicate that the task is done and the queue can move on to the next task
```

```
async def main():
```

```
    q=asyncio.Queue() # to create a queue for communication between coroutines
```

```
    await asyncio.sleep(1)
```

```
    t1=asyncio.create_task(put_data(q))
```

```
    t2=asyncio.create_task(get_data(q))
```

```
await t1,t2
```

```
print("The End")
```

```
asyncio.run(main())
```

```
'''
```

```
#
```

```
# Date : 17-04-2026
```

```
#
```

```
# Date : 18-04-2026
```

```
# saturday
```

```
#
```

```
# Date : 15-04-2026
```

```
# sunday -- holiday
```